

Prácticas Algoritmia

Saúl López Díaz

UO307112



Curso 2025-2026

Grupo: L2

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

INDICE

PRÁCTICA 0-----	2
PRÁCTICA 1.1-----	4
PRÁCTICA 1.2-----	8
PRÁCTICA 2-----	11
PRÁCTICA 3-----	13
PRÁCTICA 4-----	16
PRÁCTICA 5-----	17
PRÁCTICA 6-----	19
PRÁCTICA 6EXTRA-----	20
PRÁCTICA 7-----	21

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

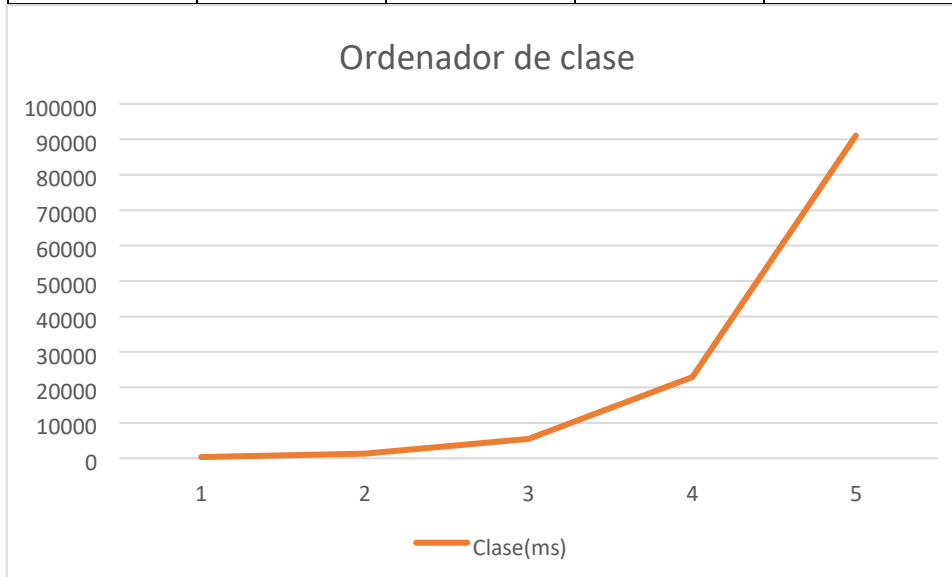


Escuela de
Ingeniería
Informática
Universidad de Oviedo



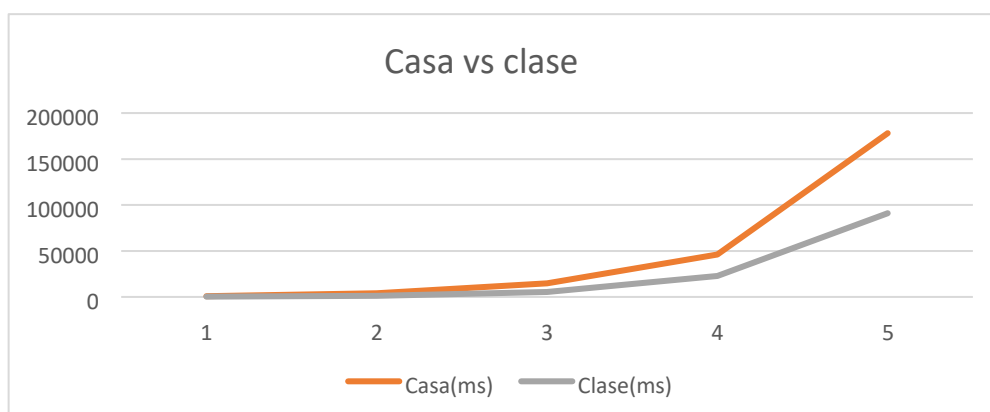
Práctica 0

N	5000	10000	20000	40000	80000
Tiempo(ms)	347	1346	5479	22874	91065



N	5000	10000	20000	40000	80000
Tiempo(ms)	807	3943	14930	46041	178091

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	



Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 1.1

Especificaciones PC de casa:

Procesador	AMD Ryzen 5 3400G with Radeon Vega Graphics	3.70 GHz
RAM instalada	16,0 GB	
Almacenamiento	447 GB SSD KINGSTON SA400S37480G, 932 GB HDD ST1000DM010-2EP102	
Tarjeta gráfica	AMD Radeon RX 7600 (8 GB)	

Ejercicio 1

Podríamos seguir utilizando esta forma de contar durante $5,8 * 10^6$ -56 años. Se le resta 56 porque hace 56 que se utiliza

Ejercicio 2

Resultados obtenidos con el pc de clase:

n	1000000	10000000	50000000	100000000	200000000
Tiempo(ms)	5	50	239	478	1098

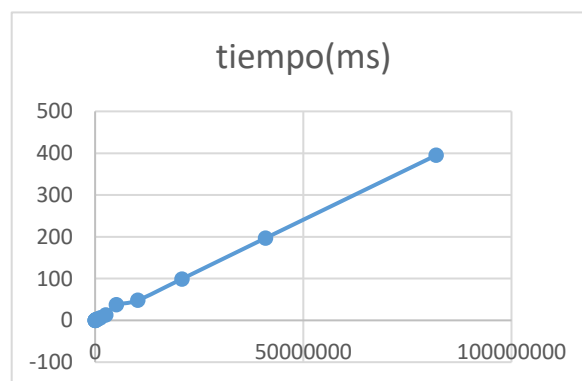
Resultados obtenidos con el pc de casa:

n	1000000	10000000	50000000	100000000	200000000
Tiempo(ms)	10	72	372	735	1494

Respuesta: A veces el tiempo sale 0 porque justo pasa el recolector de basura y detecta que nuestro programa no hace nada, por lo que detiene la ejecución. Los resultados empiezan a ser fiables a partir de $n = 10000000$, ya que es cuando se pasa el baremo de los 50 ms

Vector 3

10000	0
20000	0
40000	0
80000	0
160000	1
320000	2
640000	3
1280000	6
2560000	13
5120000	37
10240000	48
20480000	99
40960000	197
81920000	395



Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Vector4

java -Xint p11.Vector4 1

n	1000	2000	4000	8000	16000	32000	6400	12800	25600	51200
	0	0	0	0	0	0	0	0	0	0
Tiempo(ms))	0	0	0	0	0	0	6	10	13	41

java -Xint p11.Vector4 10

n	1000	2000	4000	8000	16000	32000	6400	12800	25600	51200
	0	0	0	0	0	0	0	0	0	0
Tiempo(ms))	0	0	0	15	11	21	41	90	183	367

java -Xint p11.Vector4 100

n	1000	2000	4000	8000	16000	32000	6400	12800	25600	51200
	0	0	0	0	0	0	0	0	0	0
Tiempo(ms))	10	20	20	61	120	231	483	951	1927	3905

java -Xint p11.Vector4 1000

n	1000	2000	4000	8000	16000	32000	6400	12800	25600	51200
	0	0	0	0	0	0	0	0	0	0
Tiempo(ms))	75	155	295	590	1200	2341	4755	9597	19080	38088

java -Xint p11.Vector4 10000

n	1000	2000	4000	8000	16000	32000	6400	12800	25600	51200
	0	0	0	0	0	0	0	0	0	0
Tiempo(ms))	949	1530	3040	6100	12135	24145	5694 4	12138 8	31969 4	FdT

java -Xint p11.Vector4 100000

n	1000	2000	4000	8000	16000	32000	6400	12800	25600	51200
	0	0	0	0	0	0	0	0	0	0
Tiempo(ms))	470	1103	2060	4105	7964	15547	3219 2	FdT	FdT	FdT

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Ejercicio 4

Tabla1

n	Tsuma	Tmaximo
10000	0	0
20000	0	0
40000	0	0
80000	16	0
160000	16	16
320000	16	32
640000	47	63
1280000	93	125
2560000	188	250
5120000	375	485
10240000	753	998
20480000	1532	1947
40960000	3058	3906
81920000	6159	7703

Tabla2

n	Tcoincidencias1 (con parámetro 1)	Tcoincidencias2
10000	797	0
20000	3092	15
40000	12220	0
80000	50050	16
160000	193684	16
320000	772527	32
640000	FdT	79
1280000	FdT	140
2560000	FdT	266
5120000	FdT	547
10240000	FdT	1063
20480000	FdT	2126
40960000	FdT	4264
81920000	FdT	8735

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?

El tiempo también aumentaría considerablemente pero su complejidad no cambiaría.

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2?

El tiempo del problema aumentaría proporcionalmente a la K que multipliquemos, es decir, cuanto mayor sea la k mayor serán los tiempos medidos.

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 1.2

Tabla 1

Parámetro: 1

n	TBucle1	TBucle2	TBucle3	TBucle4
100	0	0	0	1
200	0	1	3	6
400	0	3	17	56
800	0	16	64	347
1600	1	56	279	2645
3200	0	249	1168	21346
6400	1	950	4897	167624
12800	3	4897	20513	1364020
25600	3	19536	85110	FdT
51200	8	76960	353165	FdT

Tabla 2

Parámetro: 1

n	TBucle5	TBucle6	TBucle7
100	1	8	22
200	4	67	309
400	23	613	4850
800	113	5476	74809
1600	572	46996	FdT
3200	2605	406658	FdT
6400	12339	FdT	FdT
12800	56181	FdT	FdT
25600	258107	FdT	FdT
51200	FdT	FdT	FdT

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Tabla 3

Parámetro: 100

n	TBucle1	TBucle2	t1 /t2
100	2	49	0,040816327
200	3	178	0,016853933
400	5	807	0,006195787
800	12	3053	0,00393056
1600	28	12125	0,002309278
3200	55	55344	0,000993784
6400	125	225943	0,000553237
12800	258	1043380	0,00024727
25600	541	FdT	FdT
51200	1098	FdT	FdT

Tabla 4

Parámetro 1

n	TBucle3	TBucle2	t3 / t2
100	0	0	0
200	3	1	3
400	17	3	5,666666667
800	64	16	4
1600	279	56	4,982142857
3200	1168	249	4,690763052
6400	4897	950	5,154736842
12800	20513	4897	4,188891158
25600	85110	19536	4,356572482

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

51200	353165	76960	4,588942308
-------	--------	-------	-------------

Tabla 5

Parámetro: 1

n	TBucle4 (Python)	TBucle4 (Java sin optimizar)	tPython / tJava
100	8	2	4
200	45	9	5
400	380	68	5,58823529
800	3173	538	5,89776952
1600	26005	4415	5,89014723
3200	235578	35628	6,61215898
6400	FdT	291495	FdT

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 2

Tabla 1: Algoritmo de la burbuja

n	t ordenado	t inverso	t aleatorio
10000	451	1086	900
20000	1702	4386	3680
40000	6854	17549	14469
80000	27489	70236	57744
160000	110955	286721	229095

Tabla 2: Algoritmo de Selección

n	t ordenado	t inverso	t aleatorio
10000	407	356	427
20000	1620	1477	1649
40000	6453	5891	6590
80000	25603	23606	26463
160000	104828	93535	105471

Tabla 3: Algoritmo de inserción

n	t ordenado	t inverso	t aleatorio
10000	1	677	345
20000	0	2714	1410
40000	0	10875	5572
80000	1	44221	22213
160000	3	178165	89317

Tabla 4: Algoritmo rápido

n	t ordenado	t inverso	t aleatorio
10000	23	28	38
20000	52	60	77
40000	105	128	165
80000	224	263	349
160000	469	569	744
320000	1002	1149	1577

Con parámetro 10, para tiempos mayores

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Tabla 5 : Rápido vs inserción para tiempos bajos

Parámetro 100000

n	t rapido	t insercion	t rapido/t insercion
10	119	59	2,016949153
15	201	109	1,844036697
20	301	179	1,681564246
25	369	259	1,424710425
30	425	384	1,106770833
35	528	488	1,081967213
40	650	654	0,993883792
45	747	770	0,97012987
50	838	938	0,893390192
55	934	1178	0,79286927
60	1045	1352	0,772928994
65	1162	1614	0,719950434
70	1333	1866	0,714362272
75	1422	2067	0,687953556
80	1457	2375	0,613473684
85	1544	2621	0,589088134
90	1747	2976	0,58702957
95	1815	3284	0,552679659
100	1922	3704	0,518898488

A partir de $n = 40$ es más rápido el algoritmo de quick sorting, mientras que para $n \leq 35$ es más rápido el algoritmo de inserción

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 3

Ejercicio 1

A partir de $n=8000$ peta

Tardaría 1572032746 años $43 \cdot 2^{60}$

Sustraccion4

n	tiempo
100	2
200	21
400	140
800	1112
1600	8868
3200	72071
6400	579465

Sustraccion 5

n	tiempo
30	356
32	1112
34	3257
36	9721
38	29247
40	87826
42	263461

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Division 4

n	tiempo
1000	24
2000	97
4000	371
8000	1194
16000	4831
32000	19366
64000	77926

Division 5

n	tiempo
1000	21
2000	99
4000	316
8000	1255
16000	4983
32000	19978
64000	80905

VectorSuma

(Con n=1000)

n	Tiempo Metodo1	Tiempo Metodo2	Tiempo Metodo3
100	788	2143	5928

Fibonacci

n	Tiempo Metodo1	Tiempo Metodo2	Tiempo Metodo3	Tiempo Metodo4
100	467	736	767	8149

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Procesamiento de puntos

n	Tiempo Trivial (ms)	Tiempo DyV (ms)
1000	61	10
2000	26	8
4000	60	9
8000	166	25
16000	598	57
32000	2194	81
64000	9135	93
128000	36919	189
256000	FdT	317
512000	FdT	689
1024000	FdT	1415
2048000	FdT	3483

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 4

Algoritmo de coloreamiento de un grafo

La complejidad esperada es de $O(n^2)$, sin embargo, no parece corresponderse con los tiempos tomados

n	Tiempo coloreado (ms)
4	6
8	0
16	0
32	0
64	0
100	0
128	0
256	1
512	8
1024	14
2048	16
4096	10
8192	35
16384	50
32768	74
65536	86

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 5

1. Definición del estado:

El valor de $dp[i][j]$ será True si es físicamente posible cargar los primeros i vehículos de la cola de tal forma que ocupen exactamente j metros en el carril de Babor. En caso contrario, será False. Es decir, un valor $dp[3][8]$ significa que es posible cargar 3 coches habiendo exactamente 8 metros ocupados en Babor.

2. Relación de recurrencia:

Para calcular la tabla $dp[i][j]$, definimos:

- i : coche actual.
- j : metros ocupados en babor.
- v_i : longitud del coche i .
- $S[i]$: suma de las longitudes de todos los coches hasta el i .
- L : límite del carril.

Caso base:

$dp[0][0]=\text{True}$ (0 coches ocupan 0 metros). El resto de la tabla se inicializa a False.

Transición:

El estado $dp[i][j]$ será True si se cumple al menos una de estas dos opciones del paso anterior ($i-1$):

Si lo metemos en Babor: La casilla anterior, restándole la longitud de este coche, era válida.

Condición: $j-v_i \geq 0$ y además $dp[i-1][j-v_i]==\text{True}$.

Si lo metemos en Estribor: La casilla anterior en la misma columna era válida (babor no crece) y el coche cabe en el espacio restante del otro lado.

Condición: $S[i]-j \leq L$ y además $dp[i-1][j]==\text{True}$.

Si no se puede ni a babor ni a estribor, $dp[i][j]=\text{False}$.

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

3. Complejidad Temporal:

$O(N * L)$ Porque usamos dos bucles for anidados: el principal recorre los N vehículos y el de dentro recorre los L metros del carril.

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 6

Fichero	Tiempo (ms)
test00.txt	65
test01.txt	3
test02.txt	5
test03.txt	4
test04.txt	6
test05.txt	8
test06.txt	4
test07.txt	4
test08.txt	41

Complejidad del algoritmo

La complejidad temporal de este algoritmo es de orden exponencial, rondando el $O(2^n)$ en el peor de los casos.

¿Por qué?

Porque el programa intenta probar todas las combinaciones posibles. Por cada objeto que leemos, el algoritmo comprueba si puede meterlo en cada uno de los contenedores ya abiertos o si tiene que abrir uno nuevo. Esto hace que las posibilidades se multipliquen por cada objeto extra que añadimos.

Sin embargo, en los tiempos reales el programa va bastante rápido gracias a dos cosas que hemos programado:

1. La poda: Si el programa se da cuenta de que por un camino ya está usando los mismos o más contenedores que en la mejor solución que había encontrado antes, deja de buscar por ahí y vuelve hacia atrás.
2. Ordenar los objetos: Como al principio ordenamos los objetos de mayor a menor, metemos primero los más grandes. Esto hace que los contenedores se llenen rápido y la poda actúe mucho antes para descartar los malos caminos.

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 6Extra

NReinas

Tamaño del tablero (n)	Tiempo sin optimización (ms)	Tiempo con optimización (ms)
4	0	0
6	0	0
8	2	0
10	29	5
12	677	105
14	24060	2680
16	OutOfMemoryException	OutOfMemoryException

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Práctica 7

Explicación de los Heurísticos aplicados:

Para mejorar nuestro algoritmo y convertirlo en "Ramifica y Poda", hemos usado tres trucos:

1. **Heurístico de Construcción (Ordenar los objetos):** Lo primero que hacemos es ordenar los pesos de mayor a menor. Así, al meter primero los objetos más grandes (que son los que más estorban), llenamos rápido el espacio y nos damos cuenta antes de si una combinación es imposible.
2. **Heurístico de Ramificación (Orden de exploración):** El programa siempre intenta meter el objeto actual en los contenedores que ya están abiertos. Dejamos la opción de abrir un contenedor nuevo como último recurso. Esto nos ayuda a encontrar soluciones bastante buenas desde el principio.
3. **Heurístico de Poda (Cota Inferior):** Este es el más importante. En cada paso, calculamos cuánto pesan en total los objetos que todavía nos faltan por colocar. Si dividimos ese peso restante entre la capacidad de un contenedor, sabemos el mínimo de contenedores extra que vamos a necesitar sí o sí. Si los contenedores que ya estamos usando sumados a esa previsión mínima superan o igualan a nuestro mejor récord, cortamos ese camino de raíz para no perder el tiempo buscando por ahí.

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Tiempos :

Fichero	Tiempo SIN RyP (ms)	Tiempo CON RyP (ms)
test00.txt	75	79
test01.txt	5	10
test02.txt	5	5
test03.txt	4	10
test04.txt	7	18
test05.txt	13	15
test06.txt	0	6
test07.txt	10	6
test08.txt	40	13
test09.txt	FdT	22

Llamadas recursivas :

Fichero	Llamadas SIN RyP	Llamadas CON RyP
test00.txt	8	8
test01.txt	30	13
test02.txt	42	20
test03.txt	161	17
test04.txt	166	27
test05.txt	31344	43
test06.txt	15	15
test07.txt	35	18
test08.txt	329439	66
test09.txt	10194928444	111

Algoritmia	Información del estudiante	Grupo
	UO:307112	L2
	Apellidos: López Díaz	
	Nombre: Saúl	

Conclusiones sobre mejoras :

Como se puede apreciar en los resultados, para pruebas muy pequeñas es más lenta la versión con ramifica y poda, debido a que tiene que calcular en cada iteración la suma total. Sin embargo, para una n mas grande obtenemos resultados mejores, sobre todo en el test9, que sin ramificar ni siquiera llegaba a completarse y con ramificación lo hace en apenas 22 ms.

En cuanto a las llamadas recursivas, aquí podemos apreciar una diferencia abismal entre ponerle poda y no ponérsela. En la versión con poda se reducen significativamente, por lo que es mucho mas rentable pensando en tamaños mucho más grandes.